Nth Fibonacci

AlgoExpert · Easy · Recursion

The problem

- F(1)=0, F(2)=1.
- Iterative with two variables.

Intuition

- Each number only depends on the previous two, so we don't need recursion or an array.

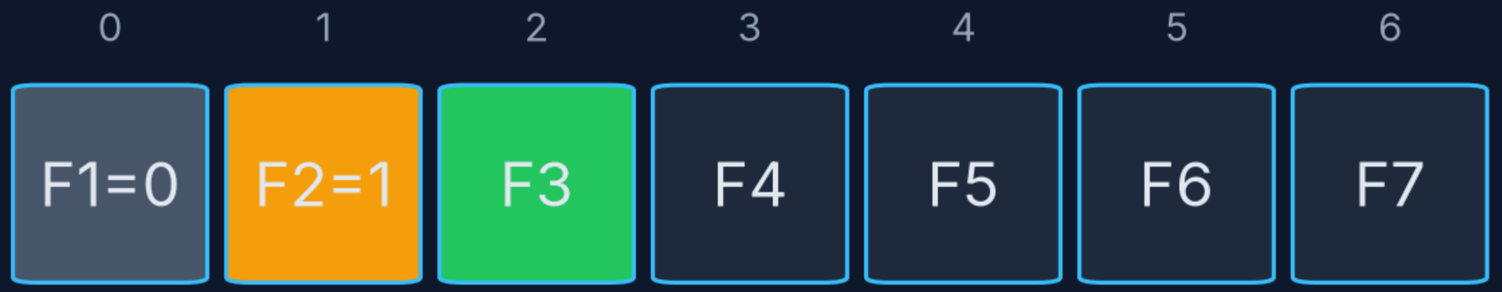
Brute force vs optimal

- Naive recursion: fib(n−1) + fib(n−2) → $O(2^n)$
- Memoisation → O(n) time, O(n) space
- Iterate with two variables → O(n) time, O(1) space

Approach

- 1. Handle n of one and two
- 2. Keep the last two values
- 3. Slide them forward n minus two times
- 4. Return the latest

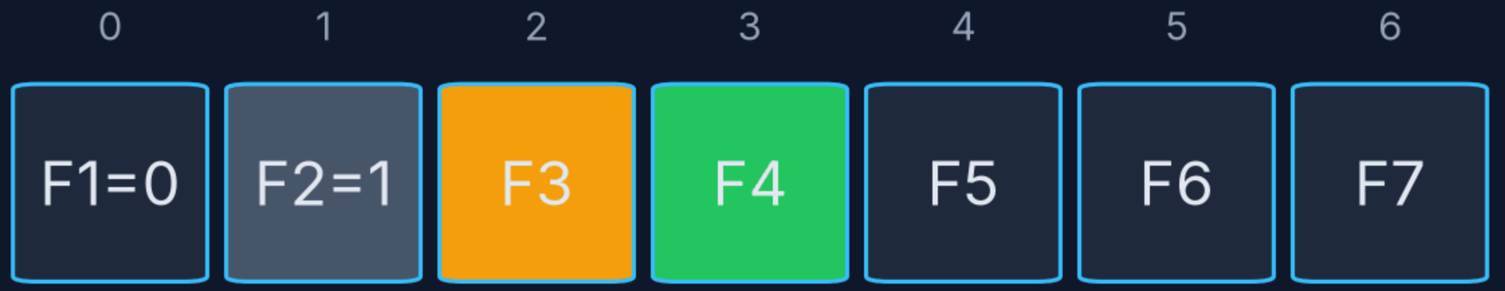
Worked example




```
last two = (1, 1)
F3 = 1
```

F3 = 1 + 0 = 1

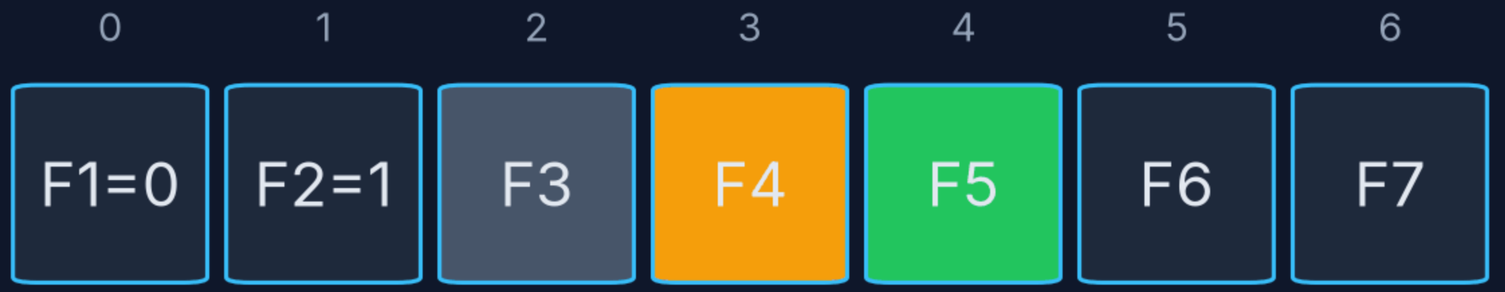
Worked example




```
last two = (1, 2)
F4 = 2
```

F4 = 1 + 1 = 2

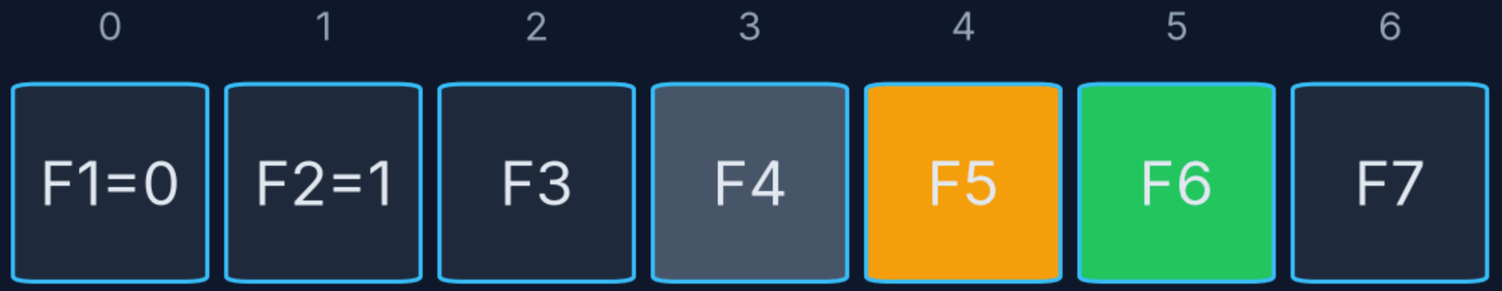
Worked example




```
last two = (2, 3)
F5 = 3
```

F5 = 2 + 1 = 3

Worked example




```
last two = (3, 5)
F6 = 5
```

F6 = 3 + 2 = 5

Worked example




```
last two = (5, 8)
F7 = 8
```

F7 = 5 + 3 = 8

Java solution

```java
public class NthFibonacci {
    public static int getNthFib(int n) {
        int a = 0, b = 1;
        if (n == 1) return 0;
        for (int i = 3; i <= n; i++) { int c = a + b; a = b; b = c; }
        return b;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- This problem is 1-indexed: F(1) = 0, F(2) = 1
- Overflow for large n (use long / BigInteger)
- N = 1 and n = 2 base cases

Interview tips & follow-ups

- Show the recursion tree to explain the blow-up
- Follow-up: O(log n) via matrix exponentiation

Recap

- Each number only depends on the previous two, so we don't need recursion or an array.
- Complexity: O(n)